GIT & GITHUB COMMANDS

Quick reference • Common commands and their uses (Git Bash)

Note: Git is the version-control tool. GitHub hosts Git repositories online. These are the commonly used commands for practicals and everyday work—not every advanced Git command.

1. Setup & configuration

Command	Use
`git --version`	Checks the installed Git version.
`git config --global user.name "Your Name"`	Sets your name for commits.
`git config --global user.email "you@example.com"`	Sets the email recorded in commits.
`git config --global --list`	Displays your Git configuration.

2. Create or download a repository

Command	Use
`git init`	Initializes a Git repository in the current folder.
`git clone REPO_URL`	Downloads a remote repository to your computer.
`git status`	Shows changed, staged, and untracked files.

3. Stage & save changes

Command	Use
`git add FILE`	Stages one file for the next commit.
`git add .`	Stages all changes in the current folder.
`git commit -m "Message"`	Saves staged changes as a commit.
`git log --oneline`	Shows a short commit history.
`git diff`	Shows unstaged changes.
`git diff --staged`	Shows staged changes not yet committed.

4. Connect and sync with GitHub

Command	Use
`git remote -v`	Lists remote repository URLs.
`git remote add origin REPO_URL`	Connects the local repo to GitHub as origin.
`git push -u origin BRANCH`	Uploads a branch; -u sets its upstream.
`git push`	Uploads new local commits to the configured remote branch.
`git pull`	Fetches and integrates changes from the remote branch.
`git fetch`	Downloads remote updates without merging them.

5. Branch commands

Command	Use
`git branch`	Lists local branches; * marks the current branch.
`git branch BRANCH`	Creates a new branch.
`git switch BRANCH`	Switches to an existing branch.

Command	Use
`git switch -c BRANCH`	Creates and switches to a new branch.
`git checkout BRANCH`	Older command to switch branches.
`git merge BRANCH`	Merges the named branch into the current branch.
`git diff BRANCH1 BRANCH2`	Shows differences between two branches.
`git diff --name-status BRANCH1 BRANCH2`	Shows only changed file names and status.
`git branch -d BRANCH`	Deletes a local branch that has been merged.

6. Undo & remove (use carefully)

Command	Use
`git restore FILE`	Discards unstaged changes in a file.
`git restore --staged FILE`	Unstages a file but keeps its edits.
`git rm FILE`	Removes a tracked file and stages the removal.
`git mv OLD NEW`	Renames/moves a tracked file.
`git revert COMMIT_ID`	Creates a new commit that reverses an earlier commit.
`git reset --soft HEAD~1`	Moves back one commit but keeps changes staged.

7. Helpful checks

Command	Use
`pwd`	Shows the current folder path in Git Bash.
`ls`	Lists files and folders.
`mkdir FOLDER`	Creates a folder.
`cd FOLDER`	Moves into a folder.
`clear`	Clears the Git Bash screen.

Mini practical workflow: create a project and push it to GitHub

1. Create/open your project folder, then run: `git init`

2. Set the main branch name if required: `git branch -M main` (or `master` for your practical).

3. Add and commit files: `git add .` → `git commit -m "Initial commit"`

4. Connect GitHub: `git remote add origin REPO_URL`

5. Push: `git push -u origin main` (replace `main` with `master` if that is your branch).

6. Create a feature branch: `git switch -c GIT-EXAM`

7. After editing: `git add .` → `git commit -m "Added program"` → `git push -u origin GIT-EXAM`

Important: Replace placeholders such as REPO_URL, FILE, BRANCH, and COMMIT_ID with your actual values. Do not type the $ prompt shown in Git Bash. For authentication, follow GitHub's browser sign-in or token/SSH setup prompts; your GitHub account password is not normally used for Git HTTPS pushes.